unrealistic (real world applications can’t afford the latency that would result in hanging clients waiting, while previous calls are being processed), this is also wasteful. We can run many copies of this pipeline, as shown in figure 7.4, but each of them will have its own cache, and will be able to handle one call at the time; caching would not be as effective as possible, because if 2 threads gets the same request, neither will be able to read the result from cache. But even worse, assuming the sentiment generator works synchronously, handling one request at the time, slows us even more.

Consider any of the threads depicted in figure 7.4, and suppose the cache has already stored the intermediate results for Twitter and Facebook when sentiment generator receives three more requests for Google, Twitter, Facebook, Google. For the first request we’ll have to compute everything from scratch, but for the other two we already have all we need in cache, and we could output the result immediately. In a synchronous architecture, however we would be forced to wait for the first call to complete before the other two (and all the others that would possibly pile up in the meantime) can be processed by the sentiment generator and returned. In an asynchronous configuration, however, the web app can call the sentiment generator asynchronously for each sentiment analysis request, “join” on the responses containing the intermediate results, and return as soon as all the info for a call is gathered and it can compute the final result.